

Module A — Travaux pratiques (énoncés étudiants)

Jour 1 — Fondations de la rétro-ingénierie & analyse de malware. Trois TP. Chacun a un **cœur guidé** (à faire par tout le monde) et un **bonus « pour aller plus loin »** (optionnel, plus difficile, sans point supplémentaire — pour ne pas s'ennuyer si on a fini en avance).

Règles de labo (à lire avant de commencer)

- On travaille **dans la VM RE-Lab** (voir [A_README_environnement.md](#)). Connexion : `vagrant ssh`.
- **Avant A2 et A3** (code « malveillant » jouet) : prendre un snapshot depuis l'hôte → `vagrant snapshot save clean`.
- Réseau **host-only** : aucune de ces cibles ne doit sortir sur Internet. Tout se passe dans `~/labs/A_re-malware/`.
- **Statique d'abord, dynamique ensuite.** On n'exécute un binaire que quand on a compris ce qu'il fait.
- On **documente au fil de l'eau** : garder un fichier `notes.md` par TP (commandes, trouvailles, IoC, hachages).
- Cadre : ces cibles sont **fournies et bénignes**, conçues pour le cours. On ne rétro-conçoit **rien d'autre**.

Les cibles sont dans `~/labs/cibles/` (déployées par le formateur). Rappel des outils : `file`, `strings`, `radare2/r2`, `objdump`, `gdb`, `ltrace`, `strace`, `yara`, `ilspycmd`.

TP A1 — Rétro-ingénierie d'un « bundle » C# (.NET)

Cible : `licensecheck` — une petite application .NET **single-file** (auto-contenue) qui demande une **clé de licence** et affiche si elle est valide.

Objectif pédagogique : constater qu'un binaire **managé** (.NET) se décompile **presque au niveau du source**, et comprendre une logique de validation à partir du code décompilé.

Cœur guidé

1. Identifier la nature du fichier.

```
cd ~/labs/A_re-malware
file *
```

Que révèle `file` ? Est-ce un ELF natif, un exécutable .NET, un « bundle » auto-extractible ? Notez vos observations.

2. Chercher les chaînes. `strings -n 6 licensecheck | less`. Repérez les messages (« licence valide/invalid »), et surtout ce qui trahit un **runtime .NET embarqué** (noms d'assemblies, `mscorlib`, `System.*`).

3. Extraire le bundle. Un exécutable .NET single-file **empaquette** les DLL managées. Objectif : récupérer l'**assembly applicative** (le `.dll` qui contient *votre* code, pas les DLL du framework). Indice :

le bundle se déballe dans un cache à l'exécution, **ou** s'inspecte avec les outils .NET. Documentez la méthode que vous utilisez.

4. **Décompiler.** Sur l'assembly applicative :

```
ilspycmd Licensecheck.dll -o ./decomp  
ls ./decomp
```

Ouvrez le code C# reconstruit. Localisez la méthode de **validation de licence**.

5. **Répondre :**

- Quel est l'**algorithme** de validation (format attendu, transformation, comparaison) ?
- Une **clé valide** est-elle calculable à la main à partir du code ? Donnez-en une et **testez-la** :

```
./licensecheck      # puis entrez votre clé
```

- Où est le **défaut de conception** ? (indice : où la vérification a lieu.)

Livrable A1 : dans `notes.md` — nature du fichier, extrait du code de validation, une clé valide trouvée par analyse, et la **leçon défensive** (2-3 lignes).

Bonus « pour aller plus loin » (optionnel, plus dur)

- **B1.a — Contournement sans clé.** Sans calculer de clé, faites accepter n'importe quelle entrée en **patchant** le binaire (inverser le branchement de la comparaison). Décrivez l'octet/instruction modifié.
Piège : sur du .NET, le patch se fait au niveau **IL**, pas de l'assembleur x86 — expliquez pourquoi.
- **B1.b — Keygen.** Écrivez un **générateur de clés** (script) qui produit une clé valide pour un nom d'utilisateur donné, en rejouant l'algorithme.
- **B1.c — Défense.** Proposez une refonte où la validation **ne peut pas** être contournée localement. Quelles hypothèses ça impose (serveur, signature) ?